

DPR.ai Frontend Modernization — Real Execution Phases

PHASE 0 — FRONTEND STABILIZATION + GOVERNANCE FOUNDATION

Purpose

Stop frontend entropy before modernization begins.

This phase exists because the frontend currently contains:

- AI-generated inconsistency
- duplicated systems
- conflicting visual languages
- layout drift
- token inconsistency
- uncontrolled abstractions
- mixed spacing systems
- giant CSS conflicts

Without this phase: all future modernization work becomes unstable.

Core Problems Being Solved

- globals.css conflicts
 - mixed spacing scales
 - hardcoded values
 - design inconsistency
 - architecture drift
 - lack of governance
 - lack of frontend rules
 - lack of AI coding standards
-

Tasks Included

Architecture Tasks

- Analyze current frontend architecture
- Identify duplicate systems
- Identify component overlap
- Identify broken abstractions
- Map current folder structure

- Map shared vs feature-specific components
- Define frontend architecture contracts

Governance Tasks

- Create frontend constitution
- Define folder ownership rules
- Define primitive/shared/domain boundaries
- Define AI coding rules
- Define token usage rules
- Define state ownership rules
- Define naming conventions
- Define import conventions

Styling Tasks

- Unify spacing scale
- Remove mixed design language
- Standardize Iron & Teal tokens
- Remove conflicting globals.css rules
- Remove glassmorphism leftovers
- Remove arbitrary Tailwind usage

Cleanup Tasks

- Identify dead files
- Identify deprecated dashboards
- Identify duplicated components
- Remove abandoned experiments

Validation Tasks

- Build stability audit
- Route rendering audit
- Visual consistency audit
- Dependency audit

PHASE 1 — FOUNDATION SYSTEM (TOKENS + PRIMITIVES)

Purpose

Build the frontend operating system.

This phase creates:

- reusable primitives
- token-driven UI
- scalable component contracts

- accessibility foundation
- density-aware UI system

This phase affects ALL 40+ pages.

Core Problems Being Solved

- inconsistent buttons
 - duplicated badges
 - typography drift
 - hardcoded colors
 - missing loading states
 - primitive inconsistency
 - accessibility inconsistency
-

Tasks Included

Token System Tasks

- Create semantic token architecture
- Create typed token exports
- Map Tailwind theme to tokens
- Define surface/canvas/border/text tokens
- Create role-aware density tokens

Typography Tasks

- Create typography scale
- Define heading/body/caption/mono styles
- Define responsive typography behavior
- Define density-aware typography

Primitive Component Tasks

- Button system
- Badge system
- Spinner system
- Icon system
- Skeleton system
- Typography primitives

Primitive Architecture Tasks

- Define primitive contracts
- Define variant strategy
- Define accessibility rules
- Define loading/error behavior
- Define sizing conventions

Testing Tasks

- Storybook setup
- Vitest snapshot tests
- Accessibility validation
- Visual regression testing

Validation Tasks

- Remove hardcoded colors
 - Ensure token-only styling
 - Ensure accessibility support
 - Ensure responsive behavior
 - Ensure primitive consistency
-

PHASE 2 — APPLICATION SHELL + NAVIGATION ARCHITECTURE

Purpose

Create the universal application frame for all 40+ pages.

This phase creates:

- layout consistency
 - role-aware navigation
 - shell governance
 - responsive structure
 - operational UX flow
-

Core Problems Being Solved

- inconsistent layouts
 - giant sidebar monolith
 - embedded role logic
 - navigation entropy
 - inconsistent page spacing
 - mobile inconsistency
-

Tasks Included

Layout Tasks

- Build PageShell
- Build Topbar

- Build Sidebar
- Build Mobile Navigation
- Build Breadcrumb system
- Build Layout wrappers

Navigation Tasks

- Create navigation config system
- Create role-aware navigation
- Create permission-based rendering
- Create navigation grouping
- Create badge/notification system

Role System Tasks

- Build RoleContext
- Build useRole hook
- Define density mapping
- Define permission system
- Define role-aware UI behavior

Responsive Tasks

- Mobile layout system
- Tablet operational layouts
- Large screen dashboard layouts
- Density-aware responsiveness

Refactor Tasks

- Split AppSidebar monolith
- Remove inline role logic
- Remove duplicated navigation logic
- Consolidate navigation rendering

Testing Tasks

- Playwright layout testing
- Role switching validation
- Responsive testing
- Navigation consistency testing

Validation Tasks

- All pages use PageShell
 - No duplicated layout logic
 - Navigation consistency
 - Mobile usability validation
-

PHASE 3 — DATA ENTRY + OPERATIONAL WORKFLOW SYSTEM

Purpose

Build the operational interaction layer.

This phase powers:

- forms
- OCR
- attendance
- tables
- workflow interactions
- operational data entry

This is the highest-usage phase in the product.

Core Problems Being Solved

- inconsistent forms
 - duplicated tables
 - poor workflow UX
 - giant OCR components
 - state chaos
 - inconsistent validation
 - missing operational density patterns
-

Tasks Included

Form System Tasks

- Input system
- Select system
- FormField wrappers
- Validation patterns
- Helper/error system
- Floor-worker variants

Data Table Tasks

- TanStack Table integration
- Virtualization support
- Sorting/filtering architecture
- Sticky headers
- Pagination system

- Empty states
- Skeleton states

OCR Tasks

- OCR input workflow
- Camera capture system
- File upload workflow
- OCR state management
- OCR review workspace
- OCR export flow

Attendance Tasks

- Attendance card system
- Tap-to-check-in workflows
- Shift indicators
- Attendance review surfaces

State Management Tasks

- Define local vs global state rules
- Define reducer usage
- Define query ownership
- Define mutation patterns
- Define optimistic updates

Refactor Tasks

- Split giant OCR page
- Remove duplicated form patterns
- Consolidate table abstractions
- Extract reusable workflow components

Testing Tasks

- Form validation tests
- OCR workflow tests
- Table interaction tests
- Mutation tests
- Error handling tests

Validation Tasks

- Workflow consistency
- Large dataset performance
- Keyboard accessibility
- Touch usability
- Form consistency

PHASE 4 — INTELLIGENCE + ANALYTICS LAYER

Purpose

Build the executive/analytical surface of the ERP.

This phase powers:

- dashboards
 - KPIs
 - charts
 - analytics
 - exports
 - operational intelligence
-

Core Problems Being Solved

- duplicated dashboards
 - inconsistent KPI design
 - chart inconsistency
 - analytics fragmentation
 - poor dashboard scalability
-

Tasks Included

Dashboard Tasks

- Dashboard consolidation
- KPI card system
- Dashboard layouts
- Stat grids
- Executive surfaces

Chart Tasks

- ApexCharts theme system
- Chart wrappers
- Shared chart configs
- Responsive chart behavior
- Chart token integration

Analytics Tasks

- Analytics surfaces
- Date range systems
- Comparative reporting
- Operational intelligence patterns

Export Tasks

- CSV export
- PDF export
- Excel export
- Export progress UX

Alert System Tasks

- Alert feed
- Severity system
- Actionable alerts
- Maintenance notifications

Refactor Tasks

- Remove competing dashboards
- Consolidate analytics components
- Standardize KPI patterns

Testing Tasks

- Dashboard rendering tests
- Chart visual regression
- Export workflow tests
- Performance profiling

Validation Tasks

- Dashboard consistency
- Responsive analytics
- Large dataset rendering
- Export reliability

PHASE 5 — FEEDBACK + POLISH + EXPERIENCE LAYER

Purpose

Make the frontend feel complete, governed, and production-ready.

This phase handles:

- overlays
- toasts
- dialogs
- auth UX
- empty states
- final interaction consistency

Core Problems Being Solved

- inconsistent feedback
 - weak auth UX
 - poor empty states
 - missing accessibility polish
 - inconsistent overlays
-

Tasks Included

Feedback System Tasks

- Toast system
- Notification patterns
- Success/error/warning/info flows
- Feedback timing rules

Modal Tasks

- Modal architecture
- Dialog system
- Focus trap support
- Accessibility behavior
- Fullscreen workflows

Overlay Tasks

- Tooltip system
- Popovers
- Context menus
- Dropdown behaviors

Empty State Tasks

- No data states
- Error states
- Permission states
- Workflow onboarding states

Auth UX Tasks

- Login redesign
- Floor-worker PIN UX
- Password recovery flows
- Session handling UX

Marketing Tasks

- Public marketing pages

- Landing page system
- Pricing surfaces
- Social proof sections

Accessibility Tasks

- Keyboard navigation
- Focus management
- Screen reader validation
- Contrast validation

Testing Tasks

- Full Playwright E2E
- Accessibility audits
- Overlay interaction testing
- Authentication flow testing

Validation Tasks

- Final UX consistency
- Accessibility compliance
- Cross-device validation
- Final frontend governance audit

FINAL PHASE — FRONTEND HARDENING + LONG-TERM SCALABILITY

Purpose

Prepare the frontend to survive years of growth and multiple contributors.

Tasks Included

Performance Tasks

- Route-based code splitting
- Lazy loading
- Suspense boundaries
- Bundle optimization
- Memoization strategy
- Virtualization audits

Scalability Tasks

- Component usage audits
- Dependency cleanup

- Dead code removal
- Shared abstraction review

Governance Tasks

- Final AI coding rules
- PR review checklist
- Component lifecycle rules
- Deprecation strategy

Testing Tasks

- Full regression suite
- Performance benchmarks
- Lighthouse audits
- Accessibility benchmarks

Documentation Tasks

- Frontend architecture docs
- Component docs
- State management docs
- Contribution guidelines

Validation Tasks

- Production readiness review
- Frontend entropy audit
- Scalability audit
- Maintainability audit